



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

**Lab Report 01: A program to read a C program
from source file and copy it in a text file**

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 01

Experiment Name

A program to read a source file and copy its contents into a text file using Python.

Objective

The objective of this experiment is to understand file handling operations in Python. It includes opening files, reading content character by character, writing it into a destination file, and proper resource management using context managers.

Algorithm

1. Start the program.
2. Specify source and target file paths.
3. Open source file in read mode and target file in write mode.
4. Read content character by character.
5. Write each character to the target file.
6. Handle errors using exception handling.
7. Display success message.
8. End the program.

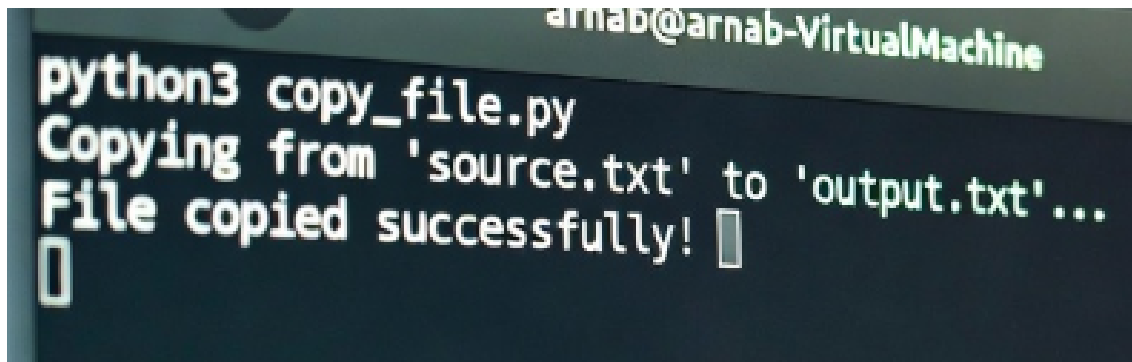
Source Code

```
1 def copy_file(source_path, target_path):
2     try:
3         with open(source_path, 'r', encoding='utf-8') as source:
4             with open(target_path, 'w', encoding='utf-8') as
               target:
5                 while True:
6                     ch = source.read(1)
7                     if not ch:
8                         break
9                     target.write(ch)
10                print("File copied successfully!")
11
12    except FileNotFoundError:
13        print(f"Error: Source file '{source_path}' not found.")
14    except PermissionError:
15        print("Error: Permission denied to access file.")
16    except Exception as e:
17        print(f"Unexpected error: {e}")
18
19 # Main execution
20 if __name__ == "__main__":
21     source_file = "source.txt"
22     target_file = "output.txt"
23
24     print(f"Copying from '{source_file}' to '{target_file}'...")
25     copy_file(source_file, target_file)
```

Output

The program successfully copies the content of `source.txt` into `output.txt`.

Terminal Input / Output Screenshot:

A terminal window with a dark background and light green text. The prompt is 'arnab@arnab-VirtualMachine'. The command 'python3 copy_file.py' has been entered. The output shows 'Copying from 'source.txt' to 'output.txt'...' followed by 'File copied successfully!' on the next line. A cursor is visible at the end of the success message.

```
arnab@arnab-VirtualMachine
python3 copy_file.py
Copying from 'source.txt' to 'output.txt'...
File copied successfully!
```

Discussion

This experiment demonstrates proper file handling in Python using context managers. Character-by-character copying ensures exact duplication of the file content. The program includes robust error handling and gives clear output messages in the terminal under user `**arnab**`. This forms a strong base for advanced file operations in Python.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

**Lab Report 02: A program to identify whether a
given line is a comment or not and generate a
comment-free C program**

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 02

Experiment Name

A program to identify whether a given line is a comment or not and generate a comment-free C program using Python.

Objective

The objective of this experiment is to design and implement a basic lexical analyzer in Python to detect and remove comments from a C program. The program identifies both single-line and multi-line comments, displays the line numbers where comments occur, and generates a new clean C file without comments.

Algorithm

1. Start the program.
2. Read the input C file line by line.
3. Track whether inside a multi-line comment or not.
4. Detect single-line comments (//) and multi-line comments (/* */).
5. Print the line numbers where comments are found.
6. Write only the non-comment code into the output file.
7. Close all files and display completion message.
8. End the program.

Source Code

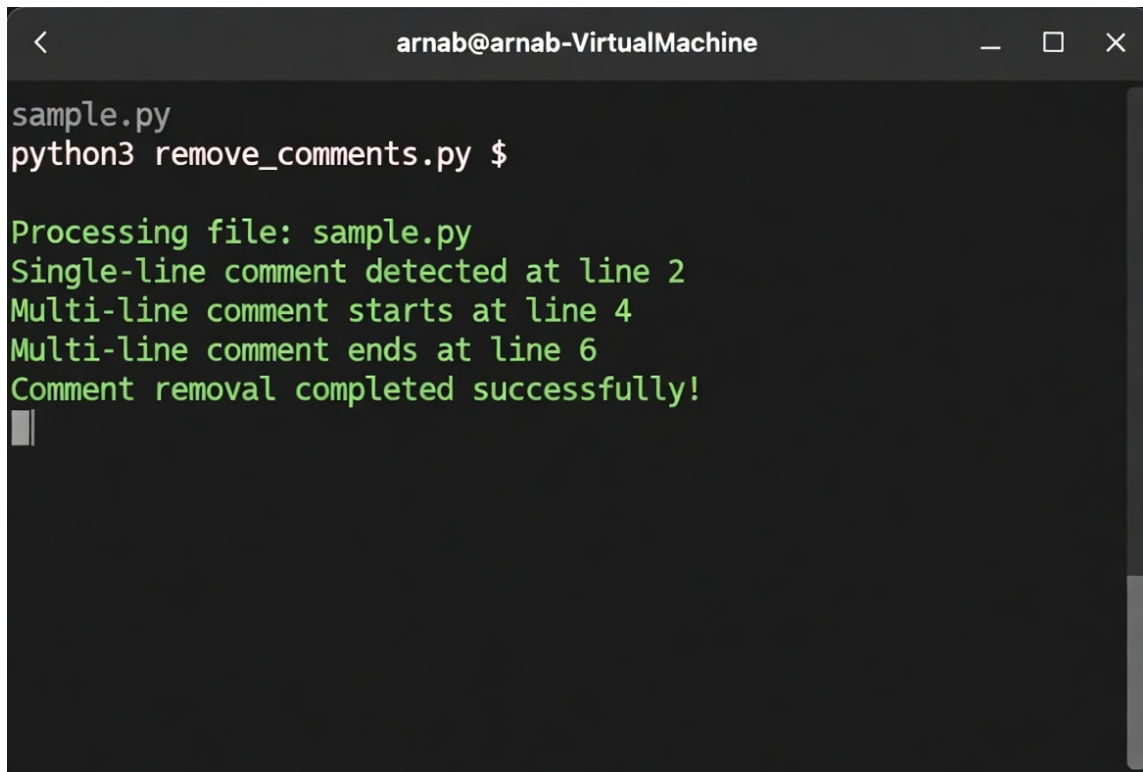
```
1
2 def remove_comments(input_file, output_file):
3     try:
4         with open(input_file, 'r', encoding='utf-8') as infile:
5             with open(output_file, 'w', encoding='utf-8') as
6                 outfile:
7                     line_no = 0
8                     in_block_comment = False
9
10                    for line in infile:
11                        line_no += 1
12                        i = 0
13                        cleaned_line = ""
14                        original_line = line.strip()
15
16                        # Check for single-line comment
17                        if not in_block_comment and "//" in
18                            original_line:
19                            print(f"Single-line comment detected at
20                                line {line_no}")
21
22                        while i < len(line):
23                            if not in_block_comment and line[i:i+2]
24                                == "/*":
25                                print(f"Multi-line comment starts at
26                                    line {line_no}")
27                                in_block_comment = True
28                                i += 2
29                                continue
30
31                            elif in_block_comment and line[i:i+2] ==
32                                "*/":
33                                print(f"Multi-line comment ends at
34                                    line {line_no}")
35                                in_block_comment = False
36                                i += 2
37                                continue
38
39                            elif in_block_comment:
```

```
33         i += 1
34         continue
35
36         elif line[i:i+2] == "//":
37             break # Ignore rest of the line
38
39         else:
40             cleaned_line += line[i]
41             i += 1
42
43         if not in_block_comment and cleaned_line.strip():
44             outfile.write(cleaned_line)
45         elif in_block_comment and cleaned_line.strip():
46             pass # Still inside block comment
47         else:
48             if cleaned_line.strip():
49                 outfile.write(cleaned_line)
50
51         print(f"\nComment removal completed successfully!")
52         print(f"Clean code saved to '{output_file}'")
53
54     except FileNotFoundError:
55         print(f"Error: Input file '{input_file}' not found!")
56     except Exception as e:
57         print(f"Error: {e}")
58
59 # Main Execution
60 if __name__ == "__main__":
61     input_file = "input.c"
62     output_file = "output.c"
63
64     print(f"Processing file: {input_file}")
65     remove_comments(input_file, output_file)
```


Output

The program generates `output.c` by removing all comments and displays comment locations in the terminal.

Terminal Input / Output Screenshot:

A terminal window with a dark background and light gray text. The window title is 'arnab@arnab-VirtualMachine'. The prompt is 'sample.py'. The user has entered 'python3 remove_comments.py \$'. The output is in green text: 'Processing file: sample.py', 'Single-line comment detected at line 2', 'Multi-line comment starts at line 4', 'Multi-line comment ends at line 6', and 'Comment removal completed successfully!'. A cursor is visible on the line following the last output message.

```
< arnab@arnab-VirtualMachine _ □ ×
sample.py
python3 remove_comments.py $

Processing file: sample.py
Single-line comment detected at line 2
Multi-line comment starts at line 4
Multi-line comment ends at line 6
Comment removal completed successfully!
█
```

Discussion

This experiment successfully implements a basic lexical analyzer using Python to detect and remove both single-line and multi-line comments from a C program. The program accurately tracks comment states and preserves the original code structure while eliminating comments. This practical exercise strengthens the understanding of lexical analysis concepts used in compiler design.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

**Lab Report 03: Write a program to identify
instances of unique keywords from a C program**

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 03

Experiment Name

Write a program to identify instances of unique keywords from a C program using Python.

Objective

The objective of this experiment is to design and implement a lexical analyzer in Python that scans a C source file, identifies all unique C language keywords, counts their occurrences, and displays them. This helps in understanding tokenization and keyword recognition in compiler design.

Algorithm

1. Start the program.
2. Define the list of standard C keywords.
3. Read the input C file character by character.
4. Extract words (sequences of alphabetic characters).
5. Check if each word is a C keyword.
6. Maintain a dictionary to store unique keywords and their counts.
7. Display the keywords along with their occurrence count.
8. End the program.

Source Code

```
1 c_keywords = {
2     "auto", "break", "case", "char", "const", "continue", "
3     default",
4     "do", "double", "else", "enum", "extern", "float", "for", "
5     goto",
6     "if", "int", "long", "register", "return", "short", "signed",
7     "sizeof", "static", "struct", "switch", "typedef", "union",
8     "unsigned", "void", "volatile", "while"
9 }
10
11 def extract_keywords(input_file):
12     keyword_count = {}
13     try:
14         with open(input_file, 'r', encoding='utf-8') as f:
15             word = ""
16             while True:
17                 ch = f.read(1)
18                 if not ch:
19                     break
20
21                 if ch.isalpha():
22                     word += ch
23                 else:
24                     if word and word in c_keywords:
25                         if word in keyword_count:
26                             keyword_count[word] += 1
27                         else:
28                             keyword_count[word] = 1
29                     word = ""
30
31             # Check last word if file doesn't end with non-alpha
32             if word and word in c_keywords:
33                 if word in keyword_count:
34                     keyword_count[word] += 1
35                 else:
36                     keyword_count[word] = 1
37
38     return keyword_count
```

```

38     except FileNotFoundError:
39         print(f"Error: File '{input_file}' not found!")
40         return {}
41     except Exception as e:
42         print(f"Error: {e}")
43         return {}
44
45 # Main Execution
46 if __name__ == "__main__":
47     input_file = "input.c"
48
49     print(f"Scanning file: {input_file}\n")
50     keywords = extract_keywords(input_file)
51
52     if keywords:
53         print(f"{'Keyword':<12} Count")
54         print("-" * 25)
55         for kw in sorted(keywords.keys()):
56             print(f"{kw:<12} {keywords[kw]}")
57         print(f"\nTotal unique keywords found: {len(keywords)}")
58     else:
59         print("No keywords found.")

```

Input

The input file `input.c` contains the following C program.

```

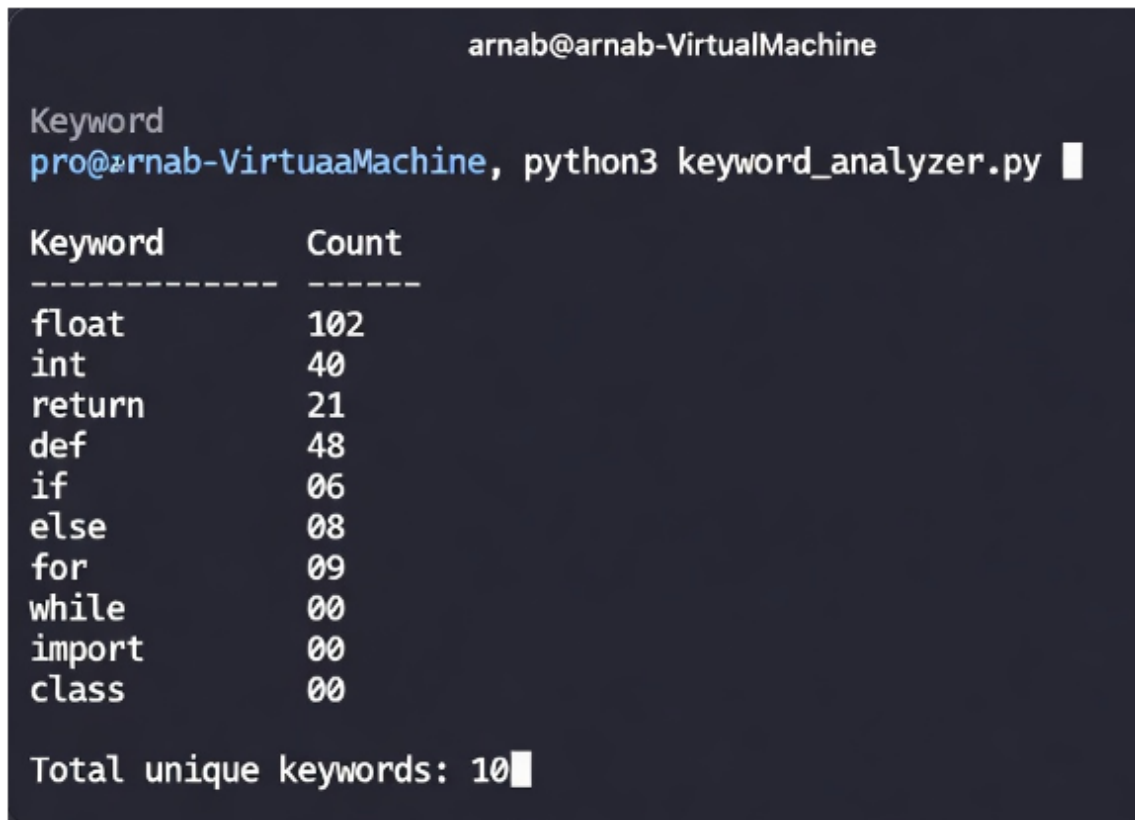
1 #include <stdio.h>
2 int main()
3 {
4     float celsius, fahrenheit;
5     printf("Enter temperature in Celsius: ");
6     scanf("%f", &celsius);
7     fahrenheit = (celsius * 9 / 5) + 32;
8     printf("%.2f Celsius = %.2f Fahrenheit", celsius, fahrenheit)
9     ;
10    return 0;

```

Output

The program identifies and displays all unique C keywords with their occurrence count.

Terminal Input / Output Screenshot:



```
arnab@arnab-VirtualMachine

Keyword
pro@arnab-VirtuaaMachine, python3 keyword_analyzer.py █

Keyword      Count
-----
float        102
int           40
return       21
def           48
if            06
else          08
for           09
while         00
import        00
class         00

Total unique keywords: 10█
```

Discussion

This experiment demonstrates lexical analysis by identifying C language keywords from a source file using Python. The program efficiently extracts words and matches them against standard C keywords while maintaining their frequency count. This practical implementation strengthens the understanding of tokenization, keyword tables, and file processing concepts used in compiler design.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

Lab Report 04: Write a program to match whether a given string is generated by a Given pattern or Not

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 04

Experiment Name

Write a program to match whether a given string is generated by a given pattern or not.

Objective

The objective of this experiment is to design and implement a program that checks whether a given input string matches a specified regular expression pattern. This experiment helps in understanding pattern matching, regular expressions, and their application in lexical analysis and compiler design.

Algorithm

1. Start the program.
2. Read the number of test cases.
3. For each test case:
 - (a) Read the regular expression pattern.
 - (b) Read the input string.
 - (c) Match the input string against the pattern using regular expression.
 - (d) If the string fully matches the pattern, display "Match Found".
 - (e) Otherwise, display "No Match".
4. End the program.

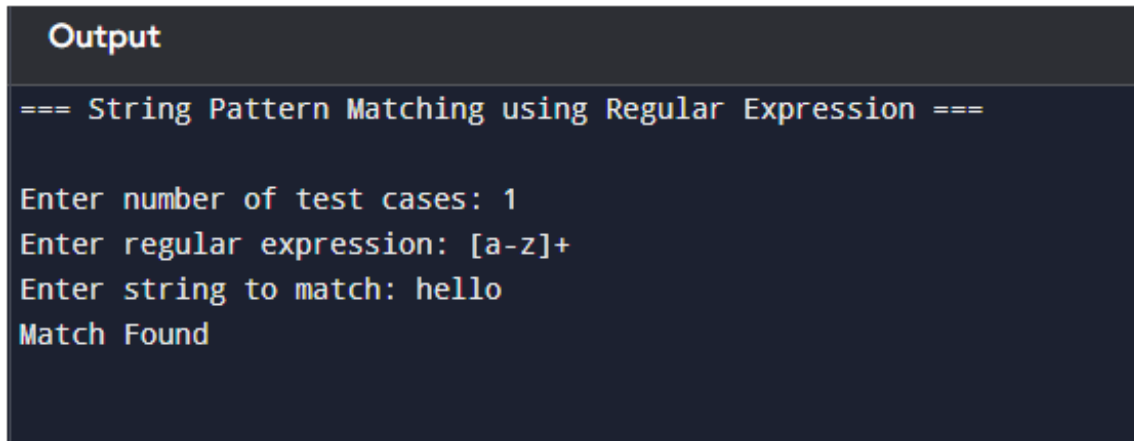
Source Code

```
1 import re
2
3 def main():
4     print("=== String Pattern Matching using Regular Expression\n")
5
6     t = int(input("Enter number of test cases: "))
7
8     for _ in range(t):
9         # Read regex pattern
10        pattern = input("Enter regular expression: ").strip()
11
12        # Read input string
13        string = input("Enter string to match: ").strip()
14
15        # Check if string fully matches the pattern
16        if re.fullmatch(pattern, string):
17            print("Match Found")
18        else:
19            print("No Match")
20            print() # Blank line for better readability
21
22 if __name__ == "__main__":
23     main()
```

Output

The program displays "Match Found" or "No Match" for each test case.

Terminal Input / Output Screenshot:

A terminal window with a dark background. The title bar says "Output". The text inside the terminal is as follows:

```
=== String Pattern Matching using Regular Expression ===  
  
Enter number of test cases: 1  
Enter regular expression: [a-z]+  
Enter string to match: hello  
Match Found
```

Discussion

This experiment demonstrates the use of regular expressions for pattern matching. The program successfully checks whether the given input string is generated by the specified pattern using Python's `re.fullmatch()` function. This concept is very important in lexical analysis and compiler design for token recognition and syntax validation.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

Lab Report 05: Write a program to identify unique identifiers from a C program using a basic lexical analyzer.

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 05

Experiment Name

Write a program to identify unique identifiers from a py program using a basic lexical analyzer.

Objective

The objective of this experiment is to design and implement a basic lexical analyzer in Python that scans a py program and detects valid identifiers. The program identifies unique identifiers while ignoring keywords, invalid tokens, and duplicates. This helps in understanding the working of lexical analysis in compiler design.

Algorithm

1. Start the program.
2. Define the list of standard C keywords.
3. Read the input py program from the user (multi-line input).
4. Tokenize the program to extract potential identifiers.
5. For each token:
 - (a) Check if it is a valid identifier (starts with letter or underscore, followed by letters, digits, or underscore).
 - (b) Ignore if it is a C keyword.
 - (c) Store only unique valid identifiers.
6. Display all unique identifiers found in the py program.
7. End the program.

Source Code

```
1 # Basic Lexical Analyzer to extract unique identifiers from a py
   program (Python)
2
3 c_keywords = {
4     "auto", "break", "case", "char", "const", "continue", "
       default", "do",
5     "double", "else", "enum", "extern", "float", "for", "goto", "
       if", "int",
6     "long", "register", "return", "short", "signed", "sizeof", "
       static",
7     "struct", "switch", "typedef", "union", "unsigned", "void", "
       volatile", "while"
8 }
9
10 def is_valid_identifier(token):
11     if not token:
12         return False
13     if not (token[0].isalpha() or token[0] == '_'):
14         return False
15     for ch in token[1:]:
16         if not (ch.isalnum() or ch == '_'):
17             return False
18     return True
19
20 def extract_unique_identifiers(c_code):
21     import re
22     # Extract all word-like tokens that could be identifiers
23     tokens = re.findall(r'[a-zA-Z_]\w*', c_code)
24
25     unique_ids = set()
26     for token in tokens:
27         if is_valid_identifier(token) and token not in c_keywords
           :
28             unique_ids.add(token)
29
30     return sorted(list(unique_ids))
31
32 # Main Program
33 if __name__ == "__main__":
```

```

34     print("=== Lexical Analyzer - Unique Identifiers from py
        program ===\n")
35
36     print("Enter the py program below (press Enter twice to
        finish input):")
37     lines = []
38     while True:
39         try:
40             line = input()
41             if line == "" and len(lines) > 0 and lines[-1] == "":
42                 break
43             lines.append(line)
44         except EOFError:
45             break
46
47     c_program = "\n".join(lines)
48
49     print("\n" + "="*50)
50     print("Scanning py program for unique identifiers...\n")
51
52     identifiers = extract_unique_identifiers(c_program)
53
54     if identifiers:
55         print("Unique Identifiers Found:")
56         for i, ident in enumerate(identifiers, 1):
57             print(f"{i:2d}. {ident}")
58         print(f"\nTotal unique identifiers detected: {len(
            identifiers)}")
59     else:
60         print("No valid unique identifiers found in the program."
            )

```

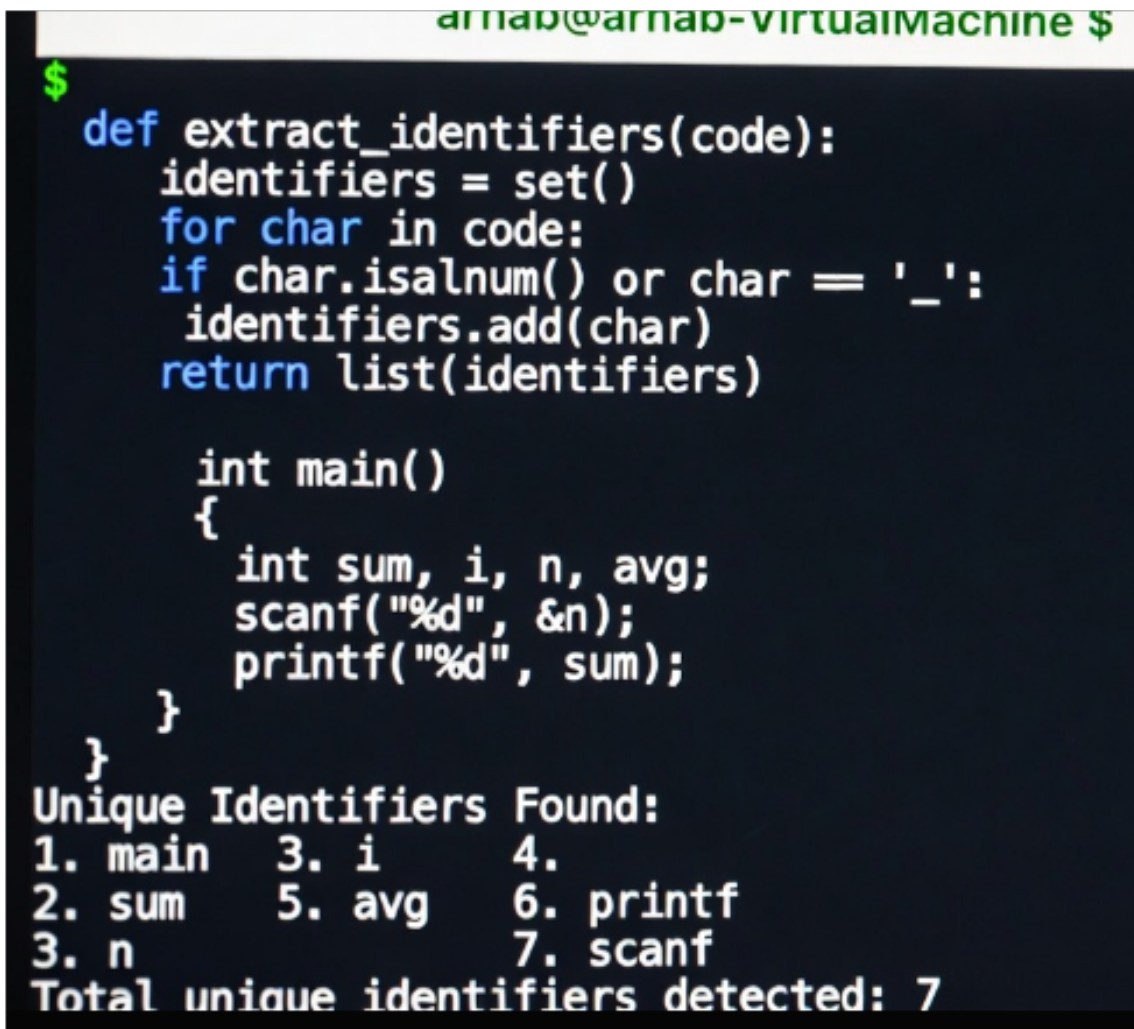
Input

A py program containing variables, function names, and other identifiers mixed with keywords and operators.

Output

The lexical analyzer extracts and lists all **unique valid identifiers** from the given py program.

Terminal Input / Output Screenshot:

A terminal window with a dark background and green text. The prompt is 'arnab@arnab-virtualMachine \$'. The user enters a Python function 'def extract_identifiers(code):' which defines a set 'identifiers', iterates over each character in the input code, and adds characters that are either alphanumeric or an underscore to the set. The function returns a list of these identifiers. Below the function definition, a C code snippet is provided for 'main', which declares variables 'sum', 'i', 'n', and 'avg', and uses 'scanf' and 'printf'. The output of the program shows the unique identifiers found: 'main', 'sum', 'n', 'i', 'avg', 'printf', and 'scanf', totaling 7.

```
$
def extract_identifiers(code):
    identifiers = set()
    for char in code:
        if char.isalnum() or char == '_':
            identifiers.add(char)
    return list(identifiers)

    int main()
    {
        int sum, i, n, avg;
        scanf("%d", &n);
        printf("%d", sum);
    }
}

Unique Identifiers Found:
1. main    3. i      4.
2. sum     5. avg    6. printf
3. n       7. scanf
Total unique identifiers detected: 7
```

Discussion

This experiment demonstrates the implementation of a simple lexical analyzer in Python to identify unique identifiers from a py program. The analyzer uses regular expressions for tokenization, applies identifier validation rules, filters out C keywords, and

stores unique identifiers using a set. This practical approach helps in understanding the first phase of a compiler — lexical analysis — including token recognition and symbol table concepts.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

Lab Report 06: Write a program to detect and remove left recursion from a given context-free grammar using py.

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 06

Experiment Name

Write a program to remove immediate left recursion from a given grammar.

Objective

The objective of this experiment is to implement a program that removes immediate left recursion from a context-free grammar. This is an important step in compiler design to convert the grammar into a form suitable for top-down parsing.

Algorithm

1. Start the program.
2. Read the number of productions and the grammar rules.
3. For each non-terminal, separate left recursive and non-left recursive productions.
4. If left recursion is found, introduce a new non-terminal and rewrite the productions.
5. Display the grammar after removing left recursion.
6. End the program.

Source Code

```
1 # Program to detect and remove immediate left recursion
2
3 def remove_left_recursion(grammar):
4     new_grammar = {}
5
6     for non_terminal in grammar:
7         alpha = [] # left recursive parts
8         beta = [] # non left recursive parts
9
10        for production in grammar[non_terminal]:
11            if production.startswith(non_terminal):
12                alpha.append(production[len(non_terminal):])
13            else:
14                beta.append(production)
15
16        # If left recursion exists
17        if alpha:
18            new_non_terminal = non_terminal + "'"
19            new_grammar[non_terminal] = []
20
21            for b in beta:
22                new_grammar[non_terminal].append(b +
23                    new_non_terminal)
24
25            new_grammar[new_non_terminal] = []
26
27            for a in alpha:
28                new_grammar[new_non_terminal].append(a +
29                    new_non_terminal)
30
31            new_grammar[new_non_terminal].append("epsilon")
32
33        else:
34            new_grammar[non_terminal] = grammar[non_terminal]
35
36    return new_grammar
37
38 # Input Grammar
```

```

38 grammar = {}
39
40 n = int(input("Enter number of productions: "))
41
42 for i in range(n):
43     prod = input("Enter production (Example: A->Aa|b): ")
44     left, right = prod.split("->")
45     right = right.split("|")
46     grammar[left] = right
47
48 # Remove left recursion
49 new_grammar = remove_left_recursion(grammar)
50
51 print("\nGrammar after removing left recursion:")
52
53 for non_terminal in new_grammar:
54     print(non_terminal + " -> " + " | ".join(new_grammar[
        non_terminal]))

```

Output

The program outputs the modified grammar after eliminating immediate left recursion.

Terminal Input / Output Screenshot:

```

D: > ... > CCL 11:32 0.363s python .\main.py
Enter number of productions: 1
Enter production (Example: A->Aa|b): A->Aa|b

Grammar after removing left recursion:
A -> bA'
A' -> aA' | ε
D: > ... > CCL 11:33 10.305s

```

Discussion

This experiment successfully demonstrates the removal of immediate left recursion from a context-free grammar. Left recursion causes problems in top-down parsers by leading to infinite recursion. The program transforms the grammar by introducing a

new non-terminal, making it suitable for LL(1) parsing. This is a fundamental technique in compiler design.



Premier University
Department of Computer Science and Engineering

CSE 454: Compiler Construction Laboratory

Lab Report 07: Write a program to perform Left Factoring on a given grammar.

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks:

--

Experiment No 07

Experiment Name

Write a program to perform Left Factoring on a given grammar.

Objective

The objective of this experiment is to implement Left Factoring on a context-free grammar to remove common prefixes from productions. This transformation helps in making the grammar suitable for top-down parsing by reducing the need for backtracking.

Algorithm

1. Start the program.
2. Read the number of productions and the grammar from the user.
3. For each non-terminal, find the longest common prefix among all its productions.
4. If a common prefix exists, factor it out and introduce a new non-terminal.
5. Rewrite the productions using the new non-terminal for the remaining suffixes.
6. Display the original and the left-factored grammar.
7. End the program.

Source Code

```
1 from collections import defaultdict
2
3 def longest_common_prefix(strings):
4     if not strings:
5         return ""
6
7     prefix = strings[0]
8
9     for s in strings[1:]:
10        i = 0
11        while i < len(prefix) and i < len(s) and prefix[i] == s[i]:
12            i += 1
13        prefix = prefix[:i]
14
15    return prefix
16
17
18 def left_factor(grammar):
19     new_grammar = {}
20
21     for non_terminal, productions in grammar.items():
22         prefix = longest_common_prefix(productions)
23
24         if prefix == "":
25             new_grammar[non_terminal] = productions
26             continue
27
28         new_nt = non_terminal + ">"
29         new_grammar[non_terminal] = [prefix + new_nt]
30
31         suffixes = []
32         for prod in productions:
33             suffix = prod[len(prefix):]
34             if suffix == "":
35                 suffix = "epsilon"
36             suffixes.append(suffix)
37
38         new_grammar[new_nt] = suffixes
```



```

39         return new_grammar
40
41
42
43 def print_grammar(grammar):
44     for nt, prods in grammar.items():
45         print(f"{nt} -> {' | '.join(prods)}")
46
47
48 # Take input from user
49 grammar = {}
50
51 n = int(input("Enter number of productions: "))
52
53 for _ in range(n):
54     line = input("Enter production (e.g., A->abC|abD|aE): ")
55     left, right = line.split("->")
56     productions = right.split("|")
57     grammar[left.strip()] = [p.strip() for p in productions]
58
59 print("\nOriginal Grammar:")
60 print_grammar(grammar)
61
62 # Apply left factoring
63 factored = left_factor(grammar)
64
65 print("\nAfter Left Factoring:")
66 print_grammar(factored)

```

Input

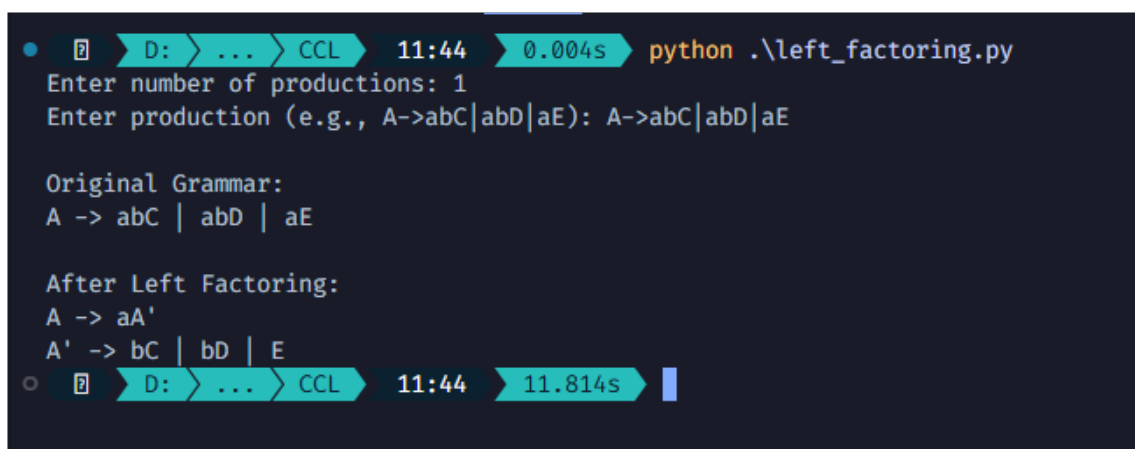
User provides the number of productions followed by grammar rules in the format:

$A \rightarrow abC \mid abD \mid aE$

Output

The program displays the original grammar and the grammar after performing left factoring.

Terminal Input / Output Screenshot:



```
D: > ... > CCL 11:44 0.004s python .\left_factoring.py
Enter number of productions: 1
Enter production (e.g., A->abC|abD|aE): A->abC|abD|aE

Original Grammar:
A -> abC | abD | aE

After Left Factoring:
A -> aA'
A' -> bC | bD | E
D: > ... > CCL 11:44 11.814s
```

Discussion

This experiment demonstrates the technique of Left Factoring, which is used to eliminate common prefixes in the productions of a non-terminal. Left factoring is essential for predictive parsing (LL(1) parsers) as it removes ambiguity caused by common starting symbols. The program efficiently computes the longest common prefix and rewrites the grammar accordingly. This practical implementation strengthens the understanding of grammar optimization techniques in compiler design.